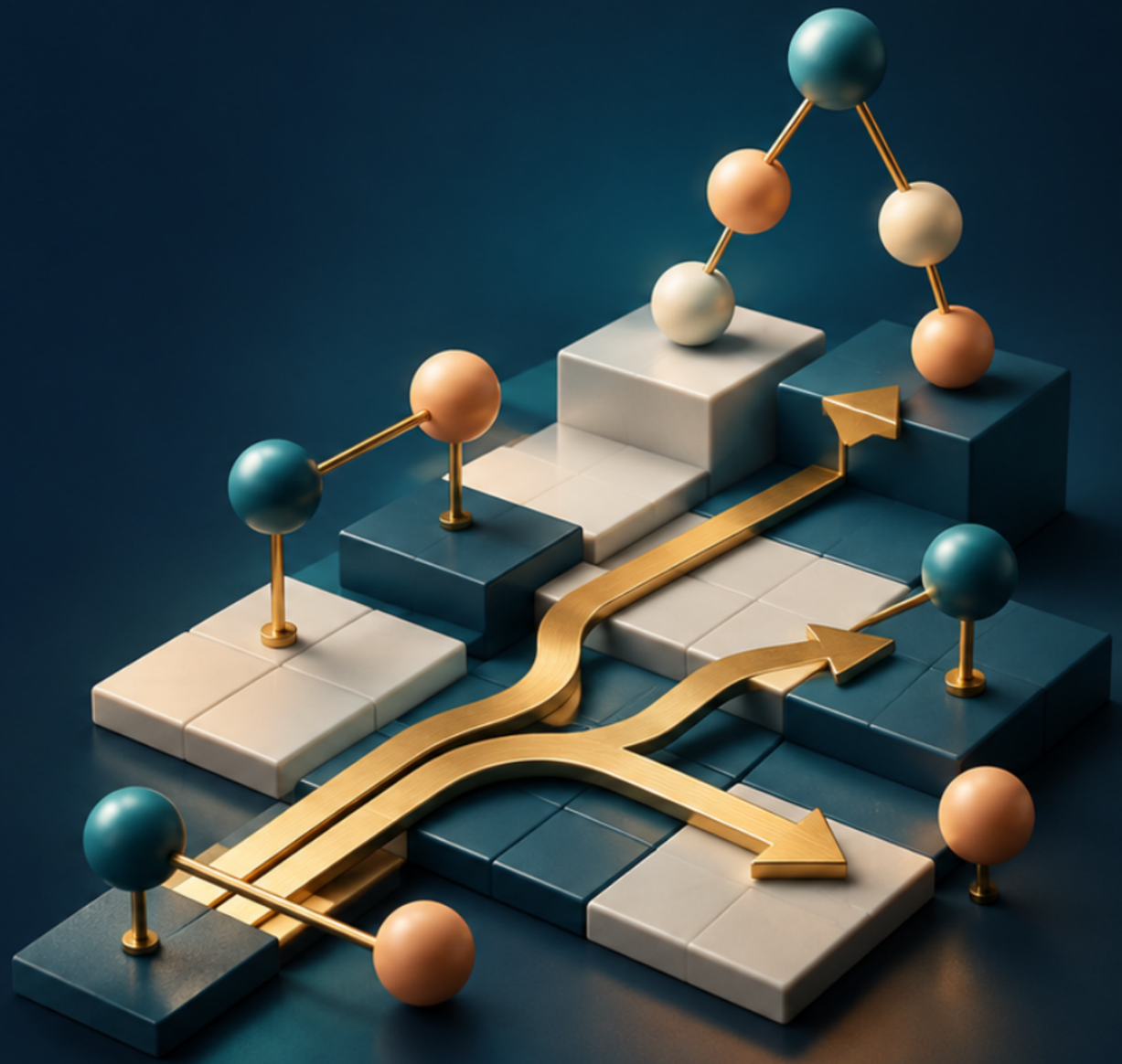
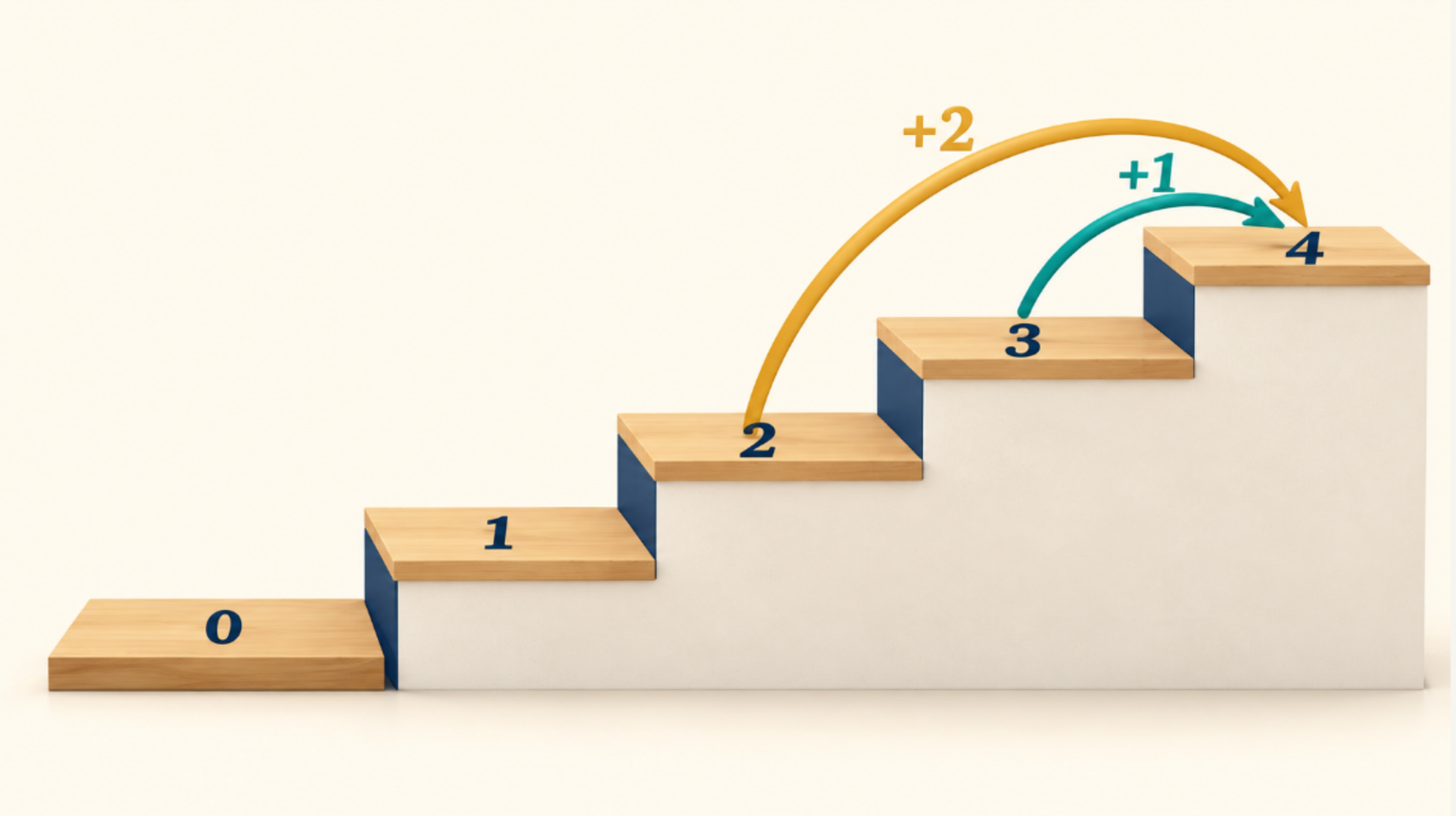
06

동적 계획법

작은 답을 구하고, 다음 계산에 이어 씁니다.



네 칸 계단을 오르는 방법



한 번에 1칸 또는 2칸.
꼭대기까지 가는 순서가
다르면 다른 방법입니다.

네 칸에서는 다섯 가지

오르는 순서	높이 합
1 + 1 + 1 + 1	4칸
1 + 1 + 2	4칸
1 + 2 + 1	4칸
2 + 1 + 1	4칸
2 + 2	4칸

답은 이동 횟수가 아니라 서로 다른 순서의 개수 5입니다.

마지막 걸음으로 두 경우를 나눈다

마지막 이동	직전 위치	그 위치까지 오는 방법
1칸 이동	3칸	1+1+1 / 1+2 / 2+1 : 3가지
2칸 이동	2칸	1+1 / 2 : 2가지

3가지 + 2가지 = 5가지

작은 계단의 답을 더한다

높이	방법 수	앞의 두 답으로 계산
1	1	1칸 한 번
2	2	1+1 또는 2
3	3	2 + 1
4	5	3 + 2
5	8	5 + 3

ways(n) = ways(n-1) + ways(n-2)

그대로 물어보면 같은 계산이 겹친다

계산하려는 답	다시 필요한 답
5칸의 방법 수	4칸의 답 + 3칸의 답
4칸의 방법 수	3칸의 답 + 2칸의 답
3칸의 방법 수	2칸의 답 + 1칸의 답

3칸의 답이 첫째 줄과 둘째 줄에서 다시 필요합니다.

답을 적어 두고 다시 읽는다

요청	저장된 답	할 일
처음 3칸을 계산	아직 없음	**계산해서 3을 저장**
다시 3칸이 필요	3이 있음	**저장한 3을 바로 사용**
4칸을 계산	3칸=3, 2칸=2	**3+2=5를 저장**

계산 결과를 기억하는 방법을 메모이제이션이라고 합니다.

해시맵은 키와 값을 짝지어 저장한다

키: 무엇의 답인가?	값: 그 질문의 답
높이 0	**방법 수 1**
높이 1	**방법 수 1**
높이 2	**방법 수 2**
높이 3	**방법 수 3**
높이 4	**방법 수 5**

Python에서는 dict, 즉 사전으로 이 표를 만듭니다.

Python 사전에 저장하고 꺼내기

코드	실행한 뒤의 뜻
memo = {0: 1, 1: 1}	0의 답 1, 1의 답 1을 처음에 저장
memo[2] = 2	키 2에 답 2를 저장
2 in memo	키 2가 있으므로 True
memo[2]	키 2에 연결된 값 2를 읽음
4 in memo	아직 키 4가 없으므로 False

in으로 키가 있는지 확인한 다음, 저장된 답을 읽습니다.

큰 질문에서 필요한 답을 찾아간다

```python
def climb_saved(n, memo):
    if n in memo:
        return memo[n]
    memo[n] = climb_saved(n-1, memo) + climb_saved(n-2, memo)
    return memo[n]

memo = {0: 1, 1: 1}
answer = climb_saved(4, memo)
```

필요한 답을 내려가며 찾고, 계산한 결과를 메모합니다.

작은 답부터 차례로 채울 수도 있다

높이 i	0	1	2	3	4	5	6
방법 수	1	1	2	3	5	8	**13**

바텀업: 0칸, 1칸에서 시작해 원하는 높이까지 올라갑니다.

6-1 계단 오르기

높이가 n칸인 계단을 바닥에서 꼭대기까지 오른다. 한 번에 1칸 또는 2칸을 오를 수 있다. 오르는 순서가 다르면 서로 다른 방법으로 센다.

01

꼭대기까지 오르는 서로 다른 방법의 수를 반환하시오.

`climb_stairs(n)`

6-1 입력과 반환값

입력 n

계단 높이. 0~90의 정수

반환값

1칸 또는 2칸씩 올라가는 서로 다른 방법의 개수

처음 두 값

dp[0]=1, dp[1]=1. n=0은 이동하지 않는 방법 한 가지

6-1 이동하지 않음 · 일곱 칸

이동하지 않음

n=0

반환값 1

빈 이동 순서 한 가지를 센다.

일곱 칸

n=7

반환값 21

5칸의 8가지와 6칸의 13가지를 더해 21가지다.

6-1 Python

```python
def climb_stairs(n):
    dp = [0] * (n+1)
    dp[0] = 1
    if n >= 1:
        dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

시간 O(n), 표 공간 O(n).

C에서 int와 long long을 고르는 이유

저장할 값	실제 크기	사용할 정수형
계단 높이 n	0~90	**int**
46칸의 방법 수	2,971,215,073 (약 29억)	**long long**
32비트 int의 최댓값	2,147,483,647 (약 21억)	**46칸의 답을 담기에는 작음**

입력은 작아도, 계산 결과는 int의 범위를 넘을 수 있습니다.

C 함수의 첫 줄을 읽는 법

long long

결과를 큰 정수형으로 반환합니다.

climb_stairs(int n)

함수 이름은 climb_stairs. 입력 높이를 n이라는 이름으로 받습니다.

{ ... } 와 return

중괄호 안이 함수가 할 일입니다. return이 결과를 돌려주며 끝냅니다.

6-1 C

```
1  long long climb_stairs(int n) {
2      long long dp[91] = {0};
3      dp[0] = 1;
4      if (n >= 1) dp[1] = 1;
5      for (int i=2; i<=n; ++i) {
6          dp[i] = dp[i-1] + dp[i-2];
7      }
8      return dp[n];
9  }
```

시간 O(n), 표 공간 O(n).

C의 기호를 말로 읽어 보기

표현	말로 읽으면
dp[0] = 1;	0번 칸에 1을 저장한다.
n >= 1	n이 1 이상인가 검사한다.
int i=2; i<=n; ++i	2번부터 시작해서 n번까지, 번호를 하나씩 늘린다.
dp[i-1] + dp[i-2]	한 칸 앞의 값과 두 칸 앞의 값을 더한다.
return dp[n];	n번 칸의 값을 답으로 돌려주고 끝낸다.

등호 = 는 저장, == 는 같은지 비교할 때 씁니다.

두 방식이 하는 계산은 같다

방식	진행 방향	답을 저장하는 때
탑다운 + 메모	큰 질문에서 필요한 작은 질문으로	**필요한 답을 계산한 뒤 저장**
바텀업 + 표	0칸, 1칸부터 원하는 높이까지	**작은 답부터 순서대로 저장**

이 수업의 네 코딩 문제는 반복문으로 푸는 바텀업을 사용합니다.

6-2 연속 부분합

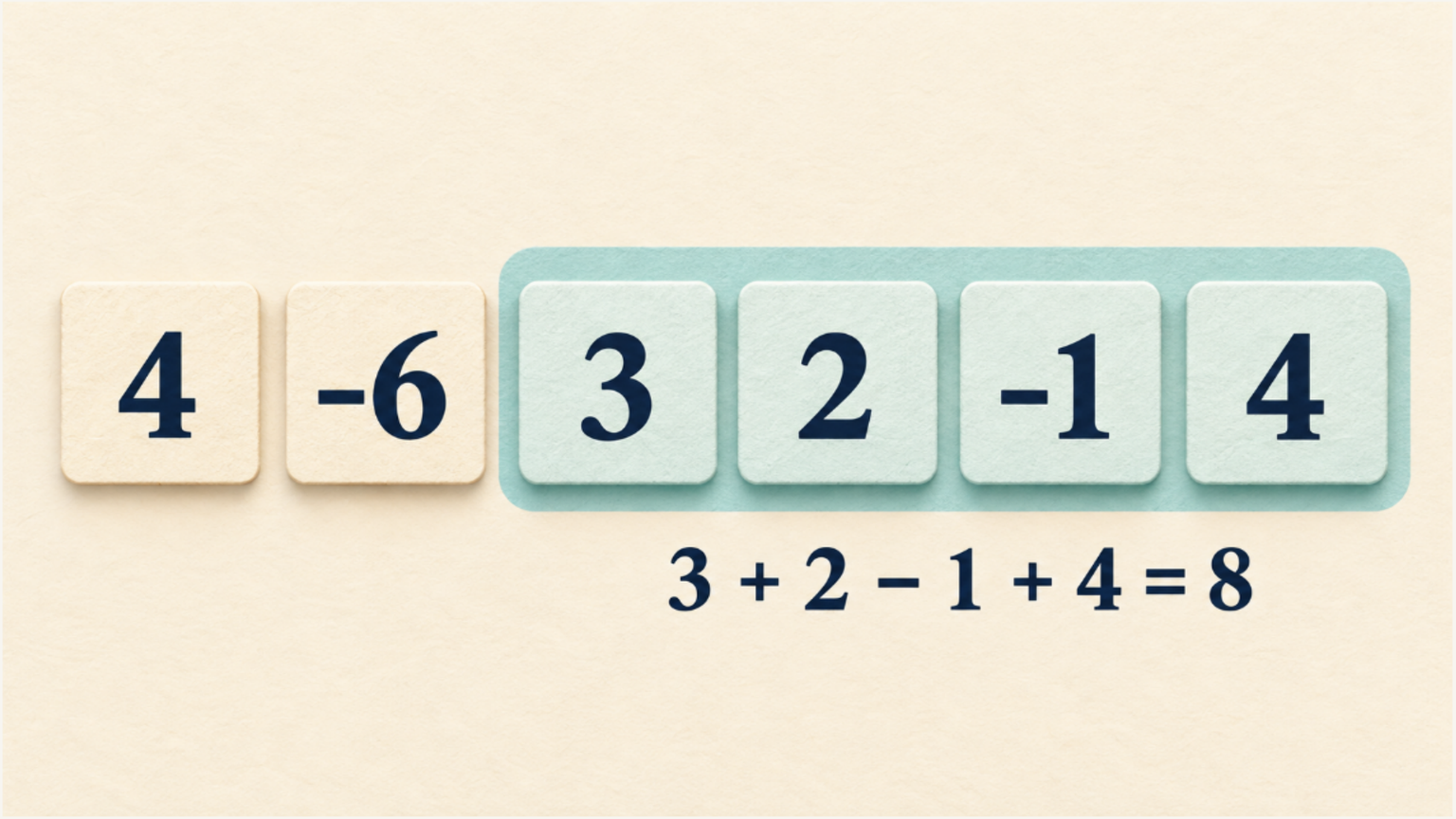
정수 배열 a에서 하나 이상의 원소가 연속해서 이어진 구간 하나를 고른다. 구간 중간의 원소를 건너뛸 수 없다.

02

고른 구간의 원소 합 중 가장 큰 값을 반환하시오.

```
max_subarray(a)
```

연속된 구간 하나를 고른다



3부터 마지막 4까지 모두 포함합니다.
합은 3+2-1+4=8입니다.

이번 수에서 끝나는 구간만 먼저 본다

끝을 세 번째 수 3으로 고정	구간 합
[4, -6, 3]	1
[-6, 3]	-3
[3]	3

세 번째 수에서 끝나는 구간의 최대 합은 3입니다.

이어 붙일까, 새로 시작할까

바로 앞에서 끝난 최대 합

4 + (-6) = -2

이번 수 3을 이어 붙이면

-2 + 3 = 1

이번 수부터 새로 시작하면

3 하나의 합 = 3. 더 큰 3을 선택

지금까지의 최고 합은 따로 기억한다

ending

이번 위치에서 반드시 끝나는 구간의 최대 합

best

지금까지 본 모든 구간의 최대 합

세 번째 수 3까지 읽었을 때

ending=3, best=4. 맨 앞의 [4]가 아직 최고

여섯 수를 끝까지 따라가 보기

이번 수	이전 ending에 붙임	새로 시작	새 ending	새 best
4	시작	4	4	**4**
-6	4-6=-2	-6	-2	**4**
3	-2+3=1	3	3	**4**
2	3+2=5	2	5	**5**
-1	5-1=4	-1	4	**5**
4	4+4=8	4	8	**8**

마지막 ending과 기존 best를 비교한 뒤, best=8을 반환합니다.

6-2 입력과 예외 조건

입력 배열 a

원소 1~10,000개. 각 값은 -1,000,000~1,000,000

반환값

하나 이상 연속해서 고른 원소들의 최대 합

처음 값

ending과 best를 첫 원소 a[0]으로 시작합니다.

6-2 모두 음수 · 원소 한 개

모두 음수

```
a=[-8,-3,-5]
```

반환값 -3

하나는 골라야 하므로 가장 큰 -3을 고른다.

원소 한 개

```
a=[-7]
```

반환값 -7

유일한 구간 [-7]의 합 -7을 반환한다.

6-2 Python

```python
def max_subarray(a):
    ending = a[0]
    best = a[0]
    for i in range(1, len(a)):
        ending = max(a[i], ending + a[i])
        best = max(best, ending)
    return best
```

시간 O(n), 추가 공간 O(1).

6-2 C 함수와 입력 배열

long long max_subarray(...)
함수가 돌려줄 최대 합은 큰 정수입니다.

const int a[], int n
a는 입력 원소들, n은 원소 수입니다. a를 읽기만 합니다.

a[0]
첫 번째 원소. 원소가 최소 하나이므로 시작값으로 쓸 수 있습니다.

6-2 C

```
long long max_subarray(const int a[], int n) {
    long long ending=a[0], best=a[0];
    for (int i=1; i<n; ++i) {
        long long extend=ending+a[i];
        if (extend>a[i]) ending=extend;
        else ending=a[i];
        if (ending>best) best=ending;
    }
    return best;
}
```

시간 O(n), 추가 공간 O(1).

모두 음수일 때 0으로 시작하면 틀린다

읽은 값	ending	올바른 best
-8	-8	**-8**
-3	-3	**-3**
-5	-5	**-3**

처음 값을 a[0]으로 잡아야 실제로 고른 구간의 합이 남습니다.

다음 계산에 필요한 정보를 남긴다

계단에서 기억한 답

해당 높이까지 오는 방법 수. 앞의 두 답을 더합니다.

연속 부분합에서 기억한 답

이번 위치에서 끝나는 최대 합 ending, 지금까지의 최고 합 best

DP를 설계할 때 먼저 할 일

저장한 숫자가 무엇을 뜻하는지 한 문장으로 정합니다.

6-3 1로 만드는 최소 연산 횟수

정수 n을 아래 연산으로 바꾸어 1을 만든다. 한 번의 연산에서는 1을 빼거나, 2·3·5 중 하나로 나눈다. 나눗셈은 나누어떨어질 때만 할 수 있다.

03

n을 1로 만드는 데 필요한 연산 횟수의 최솟값을 반환하시오.

`make_one(n)`

10에서 한 번에 갈 수 있는 수

첫 연산	다음 수	그 수에서 1까지	전체 횟수
1 빼기	9	2회	1+2=3회
2로 나누기	5	1회	1+1=2회
5로 나누기	2	1회	1+1=2회

10은 3으로 나누어떨어지지 않습니다. 최소는 2회입니다.

칸 안에 최소 연산 횟수를 적는다

저장 칸	뜻	들어갈 값
dp[1]	1에서 1까지 필요한 최소 횟수	**0**
dp[5]	5에서 1까지 필요한 최소 횟수	**1**
dp[10]	10에서 1까지 필요한 최소 횟수	**2**

dp[10]=2는 10을 1로 만드는 데 최소 두 번이 필요하다는 기록입니다.

작은 수의 최소 횟수부터 채운다

현재 수	다음 수 후보	후보의 전체 횟수	저장할 최소
1	이미 도착	0	**0**
2	1	1	**1**
3	2 / 1	2 / 1	**1**
4	3 / 2	2 / 2	**2**
5	4 / 1	3 / 1	**1**
6	5 / 3 / 2	2 / 2 / 2	**2**

다음 수는 항상 현재 수보다 작습니다. 필요한 답이 먼저 준비됩니다.

나누기 전에 1을 빼야 더 빠를 수도 있다

먼저 1 빼기

n=26

반환값 3

26–1=25, 25÷5=5, 5÷5=1로 세 번이다.

처음에 나눌 수 없음

n=7

반환값 3

7–1=6, 6÷3=2, 2÷2=1로 세 번이다.

6-3 Python

```python
def make_one(n):
    dp = [0]*(n+1)
    for x in range(2, n+1):
        dp[x] = dp[x-1]+1
        for d in (2,3,5):
            if x % d == 0:
                dp[x] = min(dp[x], dp[x//d]+1)
    return dp[n]
```

시간 O(n).

C의 나머지와 조건을 읽는 법

표현	값 또는 읽는 방법
10 % 2	나머지 0 → 2로 나눌 수 있음
10 % 3	나머지 1 → 3으로 나눌 수 없음
x % d == 0	나머지가 0과 같은지 검사
조건 A && 조건 B	두 조건이 모두 맞을 때 실행
dp[x/d] + 1	나눈 다음 수의 답에 첫 이동 한 번을 더함

C의 /는 정수끼리 나눌 때 몫을 구합니다. 먼저 나머지를 검사합니다.

6-3 C 함수와 작업 배열

int make_one(int n, int dp[])

입력 n과 계산을 기록할 배열 dp를 받습니다.

dp의 길이

n+1칸. n=10이면 0~10번의 11칸을 준비합니다.

저장할 횟수는 int

1만 계속 빼도 최대 n-1회. 이 범위에서는 충분합니다.

6-3 C

```
1   int make_one(int n, int dp[]) {
2       int divisors[]={2,3,5}; dp[1]=0;
3       for (int x=2; x<=n; ++x) {
4           dp[x]=dp[x-1]+1;
5           for (int j=0; j<3; ++j) {
6               int d=divisors[j];
7               if (x%d==0 && dp[x/d]+1<dp[x]) dp[x]=dp[x/d]+1;
8           }
9       }
10      return dp[n];
11  }
```

시간 O(n).

6-4 한 번씩만 담는 배낭

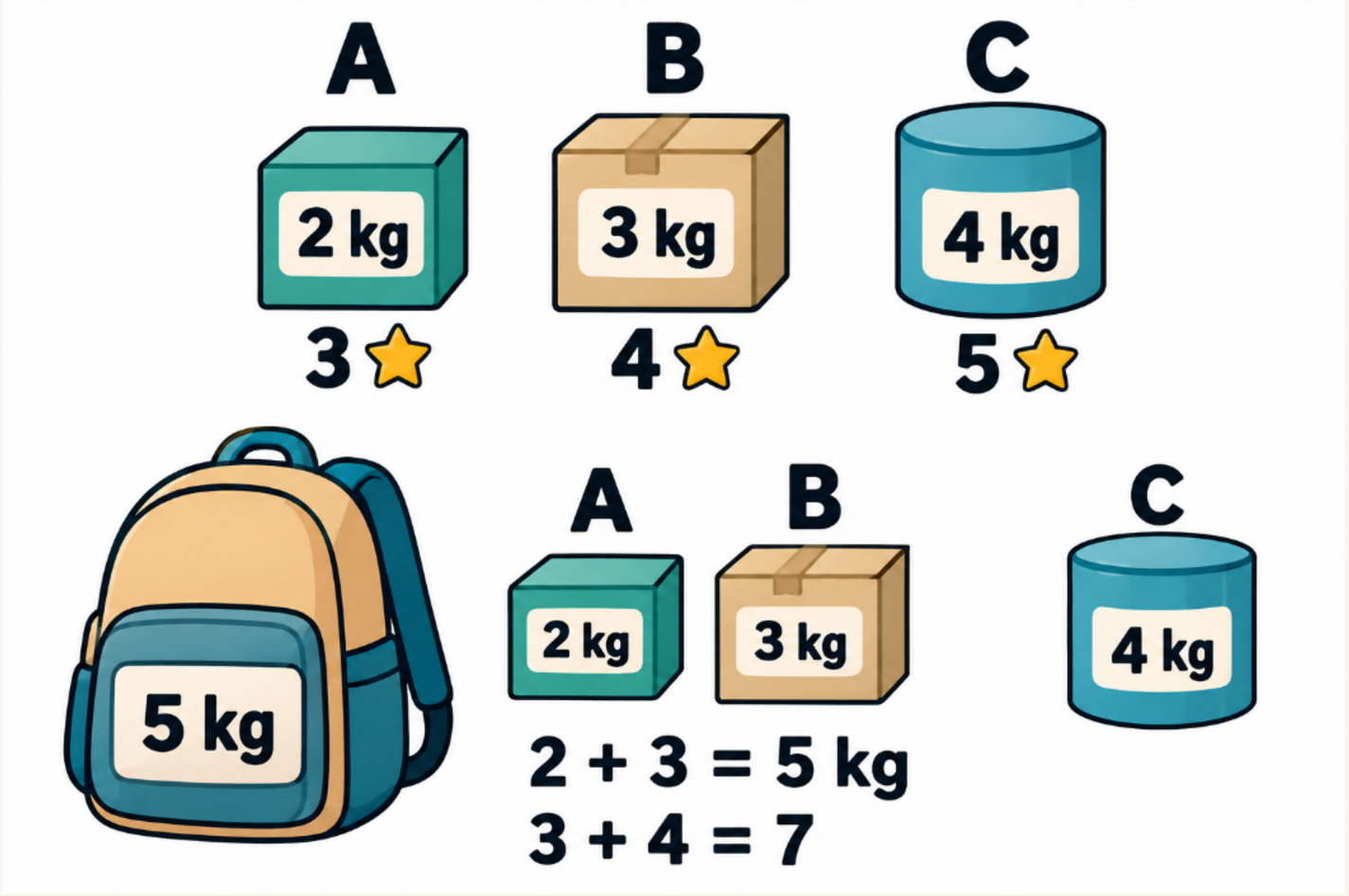
무게 제한이 W인 배낭과 여러 물건이 있다. 각 물건에는 무게와 가치가 있다. 물건마다 한 개씩만 있으며, 물건을 쪼개서 담을 수 없다. 담은 물건의 총무게는 W 이하여야 한다.

04

담을 수 있는 물건들의 가치 합 중 최댓값을 반환하시오.

```
knapsack2(items, W)
```

무게 5 안에서 가치를 가장 크게



A와 B를 함께 담으면 무게 2+3=5, 가치 3+4=7입니다.

배낭 문제에서 지킬 조건

물건 하나는 한 번만

같은 항목을 반복해서 담을 수 없고, 물건을 쪼갤 수 없습니다.

용량은 상한

총무게가 W 이하이면 됩니다. 공간이 남아도 괜찮습니다.

아무것도 담지 않음

허용하는 선택이며 가치는 0입니다.

표의 행은 물건, 열은 배낭 용량

사용할 물건	0kg	1kg	2kg	3kg	4kg	5kg
없음	0	0	0	0	0	**0**
A까지	0	0	3	3	3	**3**
A,B까지	0	0	3	4	4	**7**
A,B,C까지	0	0	3	4	5	**7**

한 칸에는 해당 물건 범위와 용량에서 얻는 최대 가치가 들어갑니다.

B를 안 담으면 윗줄의 답을 쓴다

계산할 칸	고려하는 선택	가치
A,B까지 / 용량 5	B를 담지 않음	**A까지 / 용량 5의 답 3**

안 담는 후보: dp[i-1][c]

B를 담으면 남은 용량을 읽는다

계산할 내용	숫자를 넣으면
B의 무게를 뺀 용량	**5 - 3 = 2**
A만 사용해 남은 2kg에 얻는 가치	**3점**
그 값에 B의 가치를 더함	**3 + 4 = 7점**

담는 후보: dp[i-1][c-w] + v

두 후보를 비교해 현재 칸을 채운다

선택	계산한 가치
B를 안 담음	**3**
B를 담음	**7**
더 큰 값을 현재 칸에 저장	**7**

dp[2][5] = max(3, 7) = 7

6-4 Python (1/2) · 표 준비

```python
def knapsack2(items, W):
    n = len(items)
    dp = []
    for i in range(n+1):
        dp.append([0]*(W+1))
```

물건 수+1개 행, 용량+1개 열을 모두 0으로 준비합니다.

6-4 Python (2/2) · 표 채우기

```
    for i in range(1, n+1):
        w, v = items[i-1]
        for c in range(W+1):
            dp[i][c] = dp[i-1][c]
            if w <= c:
                take = dp[i-1][c-w] + v
                dp[i][c] = max(dp[i][c], take)
    return dp[n][W]
```

시간 O(nW). 빈 선택을 허용하며 n=0 또는 W=0의 답은 0이다.

C에서는 무게와 가치를 배열로 나눠 받는다

번호	무게 배열 w	가치 배열 v
0	2	3
1	3	4
2	4	5

n=3, W=5. dp는 (n+1)행과 1001열을 가진 작업 표입니다.

6-4 C (1/2) · 표 준비와 반복

```
1  int knapsack2(const int w[], const int v[], int n, int W,
2                int dp[][1001]) {
3      for (int c=0; c<=W; ++c) dp[0][c]=0;
4      for (int i=1; i<=n; ++i) {
5          for (int c=0; c<=W; ++c) {
6              dp[i][c]=dp[i-1][c];
```

이어서 같은 반복문 안에서 담는 후보를 비교합니다.

6-4 C (2/2) · 선택과 반환

```
 7                if (w[i-1]<=c) {
 8                    int take=dp[i-1][c-w[i-1]]+v[i-1];
 9                    if (take>dp[i][c]) dp[i][c]=take;
10                }
11            }
12        }
13        return dp[n][W];
14    }
```

시간 O(nW). 빈 선택을 허용하며 n=0 또는 W=0의 답은 0이다.

6-4 남는 공간 · 물건이 없음

정확히 채우지 않아도 됨

```
items=[[2,7]], W=3
```

반환값 7

2kg만 담고 1kg를 남겨도 된다.

물건이 없음

```
items=[], W=5
```

반환값 0

담을 물건이 없으므로 가치 합은 0이다.

네 문제에서 저장한 답의 뜻

문제	저장하는 답	다음 답을 만드는 방법
계단	높이 i까지 오는 방법 수	**앞의 두 방법 수를 더함**
연속 부분합	이번 위치에서 끝나는 최대 합	**이어서 붙이기 / 새로 시작**
1로 만들기	수 x에서 1까지의 최소 횟수	**허용된 다음 수의 답+1 중 최소**
배낭	물건 범위와 용량에 따른 최대 가치	**이번 물건을 담음 / 안 담음**

한 칸 또는 변수에 담는 뜻이 다르므로, 시작값과 비교 방법도 달라집니다.

반복 계산을 줄이기 전에 확인할 것

작은 답을 정확히 정의

무엇을 입력으로 구분하고, 어떤 값을 답으로 저장할지 정합니다.

다음 계산에 충분한 정보

연속 부분합은 best만으로 부족해 ending도 기억했습니다.

준비된 답을 읽는 순서

필요한 이전 답을 먼저 구한 뒤 현재 답을 계산합니다.

6장 공개 코딩 문제 네 개

번호	함수	입력	돌려줄 값
6-1	climb_stairs(n)	계단 높이	**오르는 방법 수**
6-2	max_subarray(a)	정수 배열	**연속 구간의 최대 합**
6-3	make_one(n)	정수 n	**1까지 최소 연산 횟수**
6-4	knapsack2(items, W)	물건과 용량	**담을 수 있는 최대 가치**

작은 예제를 손으로 계산한 다음, 같은 값을 코드에서 확인해 보세요.